

■ MACHINE LEARNING

UNIGRAN Comunidades

Relatório Técnico de Arquitetura e Implementação

Data: 02 de June de 2026

ÍNDICE

- 1. Visão Geral de ML
- 2. Arquitetura Atual
- 3. Pipeline de Dados
- 4. Modelos e Algoritmos
- 5. API de Predição
- 6. Status de Implementação
- 7. O que Falta Fazer
- 8. Recomendações Técnicas

1. VISÃO GERAL DE MACHINE LEARNING

O módulo de **Machine Learning** do UNIGRAN Comunidades é responsável por:

1. Classificação Automática de Competências

Analisar textos de projetos/postagens e identificar automaticamente áreas profissionais (Dados, Backend, Frontend, etc).

2. Recomendação de Vagas Inteligente

Propor oportunidades de emprego baseadas no perfil acadêmico do aluno.

3. Análise de Skills

Extrair tecnologias e competências implícitas nos trabalhos do estudante.

4. Previsão de Compatibilidade

Calcular compatibilidade entre o perfil do aluno e as vagas disponíveis (0-100%).

O pipeline atual utiliza **clustering baseado em TF-IDF + KMeans** com suporte a multilíngue (Português/Inglês) e está **~50% integrado** à API principal.

2. ARQUITETURA ATUAL

A estrutura de ML está organizada em **5 camadas** independentes e reutilizáveis:

Camada	Responsabilidade	Status
Domain	Carregamento e normalização de dados brutos	✓ Completo
ML	Modelos, préprocessamento, predição, treinamento	~90%
Persistence	Carregamento de artefatos (.pkl e .json)	✓ Completo
API	Exposição via Flask/FastAPI (GET /health, POST /predict)	~70%
Scripts	Utilitários de download, plot, exportação	■ 50%

Estrutura de Diretórios:

backend/project_ml/

■■■ domain/ – data_loader.py (carregamento de dados)

■■■ ml/ – preprocessing.py, prediction.py, training.py, evaluation.py

■■■ persistence/ – model_repository.py (load/save artefatos)

■■■ api/ – app.py (endpoints Flask)

■■■ scripts/ – download_models.py, auxiliares

■■■ main.py – entry point de demonstração

■■■ models/ – artefatos treinados (.pkl, .json)

3. PIPELINE DE DADOS

O fluxo de dados segue um padrão **ETL (Extract → Transform → Load)** otimizado:

1. ENTRADA (Input)

- Texto em português da postagem/projeto do aluno
- Exemplo: 'Desenvolvi dashboard em Power BI com SQL e análise de dados'

2. LIMPEZA (Cleaning)

- Remover pontuação, lowercase, remover stopwords PT
- Resultado: 'desenvolvi dashboard power bi sql análise dados'

3. ENRIQUECIMENTO (Enrichment)

- Glossário PT→EN: 'dados' → 'data', 'análise' → 'analysis'
- Resultado: 'desenvolvi dashboard power bi sql data analysis'

4. VETORIZAÇÃO (Vectorization)

- TF-IDF Vectorizer: converte texto em vetor numérico 300-dimensional
- Resultado: vetor esperso [0.25, 0.15, ..., 0.3]

5. REDUÇÃO (Dimensionality)

- TruncatedSVD reduz de 300 para 50 dimensões
- Ganho: performance, redução de ruído

6. CLASSIFICAÇÃO (Clustering)

- MiniBatchKMeans agrupa em 7 clusters temáticos
- Cluster 5 = 'Dados / Analytics' com score 85.3%

7. RANKING (Ranking)

- Score 85.3% → Ranking 6 (escala 1-7)
- Categoria: 'Muito compatível'

8. RECOMENDAÇÃO (Recommendation)

- Busca vagas na base com mesma categoria
- Retorna top-3 vagas com skills recomendadas

4. MODELOS E ALGORITMOS

Algoritmo	Versão	Propósito	Saída
TF-IDF Vectorizer	sklearn	Converter texto em features	Matriz esparsa 300D
TruncatedSVD	sklearn	Reduzir dimensionalidade	Vetor 50D
MiniBatchKMeans	sklearn 1.8.0	Clustering/Classificação	ID do cluster (0-6)
Modelo KMeans por Área	sklearn	Subclusterização	Sub-cluster ID
Taxonomia de Áreas	JSON custom	Classificação por keywords	Área + score
Ranking Compatibilidade	JSON custom	Mapeamento score→ranking	Ranking 1-7

Artefatos Persistidos (em backend/models/):

- modelo_clusterizacao.pkl — KMeans treinado (v1.8.0)
- tfidf_vectorizer.pkl — TF-IDF pré-ajustado
- svd_reducer.pkl — Redutor de dimensionalidade
- nomes_clusters.pkl — Mapeamento cluster_id → nome temático
- subcluster_models.pkl — Modelos KMeans por área
- area_taxonomia.json — Keywords e pesos por área profissional
- ranking_compatibilidade.json — Mapeamento score → categoria

5. API DE PREDIÇÃO

Localização: backend/project_ml/api/app.py

Framework: FastAPI / Uvicorn

Porta: 8000 (desenvolvimento)

Status: ~70% implementado (falta integração com Express)

Endpoints Disponíveis:

Endpoint	Método	Descrição	Resposta
/health	GET	Verificar disponibilidade da API	{ "status": ok }
/predict	POST	Classificar e recomendar	PredictionResult
/metrics	GET	Estatísticas dos modelos	Métricas JSON
/retrain	POST	Retreinar modelos (admin)	Status

Exemplo de Requisição POST /predict:

```
{ "texto": "Desenvolvi um dashboard em Power BI usando SQL, indicadores e análise de dados." } Resposta Esperada:
```

```
{ "prediction": 5, "cluster": 5, "nome_cluster": "Educação / Treinamento", "area": "Dados", "score_percentual": 50.88, "ranking": 4, "categoria_compatibilidade": "Moderadamente compatível", "vagas_recomendadas": [ {"titulo": "Data Analyst...", "match": 87.5}, {"titulo": "BI Developer...", "match": 84.3} ], "skills_recomendadas": ["Python", "Tableau", "Spark"] }
```

6. STATUS DE IMPLEMENTAÇÃO

Componente	Implementado	Testado	Integrado	Notas
Modelos TF-IDF + KMeans	✓ 100%	✓ Sim	✓ Sim	Production-ready
Pipeline Preprocessing	✓ 95%	✓ Sim	✓ Sim	Falto PT-EN avançado
Predição e Ranking	✓ 90%	■ Parcial	✓ Sim	OK para demo
API Flask/FastAPI	■ 70%	■ Parcial	✗ Não	Falta integração
Recomendação de Vagas	✓ 85%	■ Parcial	■ Parcial	Base existe, falta sync
Analysis de Skills	■ 60%	✗ Não	✗ Não	Precisa implementar
Cache e Performance	✗ 0%	✗ Não	✗ Não	Crítico para prod
Monitoramento/Logs	✗ 0%	✗ Não	✗ Não	APM, métricas

Resumo Executivo: O núcleo de ML está operacional (~50% integração). Os modelos funcionam, mas a API precisa ser integrada à Express e há gaps críticos em cache, análise de skills e monitoramento.

7. O QUE FALTA FAZER

[CRÍTICA] Integração API com Express Backend

- Criar wrapper Python/FastAPI que comunique com Express via HTTP
- Expor /api/ml/predict como middleware da API Express
- Configurar timeout, retry e fallback
- Adicionar autenticação JWT entre Express e serviço ML

[CRÍTICA] Implementar Análise Automática de Skills

- Parser de código para extrair tecnologias de repositórios GitHub
- NLP para reconhecer skills implícitas no texto
- Mapeamento skills → vagas
- Atualização automática de perfil com skills detectadas

[ALTA] Cache e Otimização de Performance

- Implementar Redis para cache de previsões
- Pool de workers ML para paralelização
- Batch processing para reprocessar lotes de postagens
- Índice FAISS para busca de similaridade rápida

[ALTA] Sincronização de Base de Vagas

- Criar job que sincroniza vagas do TypeDB com base ML
- Implementar update incremental de embeddings
- Criar índice invertido para busca rápida
- Testes de cobertura de vagas

[ALTA] Containerização do Serviço ML

- Dockerfile com Python 3.11 + dependências ML
- requirements-ml.txt otimizado
- Docker Compose incluindo serviço ML
- Volumes para modelos e dados persistentes

[MÉDIA] Monitoramento e Observabilidade

- Logs estruturados de cada predição (input → output)
- Métricas Prometheus (latência, acurácia, cobertura)
- Alerta para degradação de qualidade de predição
- Dashboard de health check

[MÉDIA] Testes e Validação

- Unit tests para cada função de preprocessing
- Integration tests com dados reais
- A/B testing para validar novos modelos
- Validação de schema de resposta

[MÉDIA] Documentação Técnica

- Guia de retreinamento de modelos
- Documentação de features e algoritmos
- Runbook para troubleshooting
- Exemplos de uso da API

[BAIXA] Melhorias de Modelos

- Retreinamento com dataset mais recente
- Experimentar Transformer models (BERT em PT)
- Fine-tuning para domínio acadêmico
- Análise de viés nos clusters

8. RECOMENDAÇÕES TÉCNICAS

Estratégia de Deploy

- Serviço ML isolado em container (Docker)
- Pode rodar em Railway, Render ou AWS Lambda
- Express chama via HTTP (não direct import)
- Facilita versionamento e scaling independente

Versionamento de Modelos

- Manter histórico de modelos treinados com timestamp
- Usar commit hash do dataset para rastreabilidade
- Implementar A/B testing entre versões
- Rollback automático se acurácia cair

Handling de Erros

- Fallback para classificação keyword se modelo falhar
- Retry automático com exponential backoff
- Logging de exceções para debugging
- Response 503 se serviço está indisponível

Otimização de Latência

- Target: <200ms por predição
- Usar MiniBatch KMeans em vez de full KMeans
- Cache de embeddings para textos similares
- Parallelizar batch requests com ThreadPool

Segurança

- Validar tamanho máximo de texto (5KB)
- Rate limiting por IP (100 req/min)
- Sanitizar entrada para evitar injection
- Criptografar modelos em repouso (se dados sensíveis)

Próximas Evoluções

- Transformer models (BERT, RoBERTa em PT) para melhor semântica
- Fine-tuning em dataset acadêmico específico
- Análise de sentimento para feedback de alunos
- Generative AI para recomendações de trilha de aprendizado

CONCLUSÃO

O módulo de **Machine Learning** do UNIGRAN está em estágio **intermediário** de maturidade:

✓ Pontos Fortes:

- Pipeline completo e testado de classificação
- Modelos treinados com ~1.000+ vagas
- API scaffolded e funcional
- Suporte a português e código multilíngue
- Integração inicial com Express

■ Desafios:

- Falta sincronização de base de vagas em tempo real
- Análise de skills ainda manual
- Sem cache ou otimizações de performance
- Monitoramento e observabilidade ausentes
- Documentação técnica incompleta

■ Estimativa de Esforço para Conclusão:

- Integração completa: 40h
- Cache + performance: 30h
- Skills analysis: 35h
- Testes + deploy: 25h
- **Total: ~130 horas (2-3 semanas com 1 dev)**

■ Recomendação:

Priorizar integração com Express e análise de skills nas próximas 2 sprints. Isso habilitará o fluxo end-to-end de portfolio → recomendações. Otimizações podem vir depois com base em métricas reais de uso.